

我的小花园游戏软件

软件设计说明书

版本号：V1.0

1 软件概述

竖屏治愈系种花经营小游戏。玩家在荒地上种花，花开收获花瓣，接邻居的改造订单把一块块荒地翻成花坛，还能把花园编成阵码分享，朋友输入阵码就能进园参观。节奏慢，订单不强迫，失败不惩罚。

数据本地存储不连服务器，激励视频变现。团结引擎C#开发，文案全键值化支持多语言，逻辑按花园、订单、分享等模块拆分。

2 运行环境

项	要求
操作系统	Android 8.0及以上 / iOS 12.0及以上
运行平台	微信小游戏、抖音小游戏、移动端
游戏引擎	Unity 2021.3 LTS (团结引擎1.10.3)
编程语言	C#
运行内存	2GB以上
存储空间	100MB以上
屏幕分辨率	720x1280以上，竖屏

3 工程结构

运行时代码在 `Assets/\_Game/Scripts/Runtime/`：

```
Runtime/  
├─ Boot/ BootRunner BootSequence BootServices  
├─ Garden/ GardenModel GardenSystem GardenView SpeciesTable
```

```

├─ Orders/ OrderModel OrderSystem OrderTable OrderBoardView
├─ Share/ ShareCodec ShareView
├─ Save/ SaveHost SaveModel SaveStore SaveSystem
├─ Audio/ AudioHost AudioStems G19AudioCatalog
├─ Iaa/ IaaSystem
├─ Localize/ LocalizeService
├─ Guide/ NewbieGuide
├─ Codex/ CodexSystem CodexView
└─ UI/ HudFont PageArt

```

4 花园布局

花园是4x4网格共16个预设种植槽，按2x2象限分成4个改造区块，每区块4槽。地面初始是荒地，完成对应区块的订单后翻成花坛。

种植槽只接受预设位置，不放自由坐标，装饰和花的投放统一走槽位接口，越界请求直接拒。GardenModel 管槽位和区块状态，GardenSystem 跑生长和收获逻辑。构造时状态必须带正好16个槽，否则报参数异常。

5 花卉品种

SpeciesTable 定义4种基础花卉：

ID	品种	色系	稀有度	生长秒数	收获花瓣
SP01	蓝铃花 bluebell	蓝	普通	3.0	1
SP02	向阳花 sunburst	黄	普通	3.5	1
SP03	蔷薇心 roseheart	红	稀有	6.0	3
SP04	薰衣草雾 lavender mist	紫	普通	4.0	2

每个品种带名称、花语、故事三类文案键，走多语言文件，不硬编码。品种支持按序号和按id两种查找，越界或未知id返回空。

6 生长阶段

花的生长分阶段，SlotStage 枚举记当前阶段：空地、生长中、盛开。种下时盖 PlantedAt 时间戳，阶段数学始终是（当前时间 - 种植时间）对比 GrowSeconds：

```

//
if (nowSeconds - slots[i].PlantedAt >= def.GrowSeconds)
{
    slots[i].Stage = SlotStage.Bloom;
    slots[i].BloomedAt = nowSeconds;
    OnBloom?.Invoke(new BloomBeat(i, speciesId, BloomBeatSeconds));
}

```

Tick 用注入的时钟，既推进在线游玩也回放离线生长，因为阶段数学只依赖时间差。时钟单调保护，时间倒退直接忽略。

7 盛开反馈

盛开时触发三通道花开反馈，节拍0.3秒：变色、飘花瓣字、闪光同时播放。OnBloom 事件每槽跨过生长阈值时只抛一次，携带槽号、品种和节拍时长，表现层据此演出。

8 盛开保留窗口与枯萎

盛开不是永久的，花有保留窗口，到期枯萎回空地：

常量	值
WiltB	2.0
BloomKeepSec	GrowSeconds × 2.0
蓝铃花保留	约6秒

```
//      =      × WiltB
static double BloomKeepSec(SpeciesDef def)
=> def == null ? 0.0 : def.GrowSeconds * WiltB;
```

枯萎是柔和淡出，槽位回空地，零资源损失、不弹负面提示。离线 gap
不推进枯萎，恢复时给在开的槽重新盖时间戳，避免玩家离线回来花都没了。SweepWilt
提供显式枯萎扫描，订单板打开前先扫，保证不拿过期花判定。

9 种植与收获

Plant 是唯一的种植入口，越界索引和非空槽一律无惩罚拒绝，只有空槽、且当前选中品种有效才种下、盖种植时间戳进入生长：

```
bool Plant(int presetSlotIndex)
{
if (presetSlotIndex < 0 || presetSlotIndex >= SlotCount) return false;
SlotState slot = State.Slots[presetSlotIndex];
if (slot.Stage != SlotStage.Empty) return false;
SpeciesDef def = SpeciesTable.Get(State.SelectedSpeciesId);
if (def == null) return false;
slot.SpeciesId = def.Id;
slot.PlantedAt = Now;
slot.Stage = SlotStage.Growing;
return true;
}
```

Harvest 只有盛开槽产出，结算后槽回空、花瓣受钱包上限裁剪：

```
HarvestResult Harvest(int presetSlotIndex)
{
```

```
SlotState slot = State.Slots[presetSlotIndex];
if (slot.Stage != SlotStage.Bloom)
return new HarvestResult(false, 0, State.Petals);
int nominal = def.PetalsOnHarvest;
if (HarvestDoublePending) { nominal *= 2; HarvestDoublePending = false; }
int credited = Math.Min(WalletCap - State.Petals, nominal);
State.Petals += credited;
//
return new HarvestResult(true, credited, State.Petals);
}
```

收获有收集动画，花瓣进钱包、上限999，收获后槽位清空可再种。收获翻倍是一次性瞬态标志、不持久化，重登不留未用翻倍。SecondsUntilBloom 给生长中槽报剩余秒数。

9.1 生长加速

广告加速 SpeedupGrowing
把每个生长中槽的剩余生长窗口减半，做法是回拨成熟度、不改阶段数学：

```
//      PlantedAt
double remain = def.GrowSeconds - grown;
slots[i].PlantedAt = Now - (grown + remain * 0.5);
```

加速返回实际受助槽数，空花园或没有生长中槽时返回0，调用方不会为空花园放广告。已到窗口边缘的槽下一次Tick自然开、不重复加速。

10 离线生长

离线时花按真实时间继续生长，Tick
回放阶段数学，回来时该开的都开了。离线花瓣堆积受钱包上限约束，回来弹离线收获摘要。

11 改造订单

核心目标玩法。OrderTable 冻结12个订单，订单目标三种判定 OrderTargetKind：

判定类型	规则
ColorFamily 色系	某一色系至少 MinSlots 株盛开
Species 品种	某一品种至少 MinSlots 株盛开
DualColor 双色	两个色系都出现，合计至少 MinSlots 株

目标只按同时盛开的槽判定：生长中的槽不算，过了保留窗口的槽也不算。订单奖励3到8花瓣，难度从单目标向双目标递增。

12 订单板

订单板同时挂3个报价（BoardSize=3）。EnsureBoard在页面打开时把板补满到3个，只抽没出现过的订单，幂等：

```
void EnsureBoard()
{
    SweepExpired();
    Garden.SweepWilt();
    while (State.Board.Count < BoardSize)
    {
        OrderDef def = DrawNewDef();
        if (def == null) break;
        State.Board.Add(new OrderState { DefId = def.Id });
    }
}
```

补板前先扫过期订单、再扫枯萎，保证玩家面对的板不持有丢失单、不拿过期花判定。RefreshBoard
整板重roll，是广告位。订单板本身不做基于时间的自动刷新，只随显式调用变化，离线不动它。

13 接单与风险窗口

玩家可以接下报价，接下即承诺，可同时有多单在途。首次接单盖在线时钟戳，进入风险窗口：

常量	值
RiskK	0.3
RiskSlackSec	8秒
窗口公式	$\text{MinSlots} \times \text{GrowSeconds} \times 0.3 + 8\text{秒}$

风险窗口就是这一单的期限：在窗口内完成才算数，窗口关闭的那个刻度本身仍算窗口内（边缘保留）。严格过窗的已接单判过期、扫出板，即使后来满足了 CompleteOrder 也拒绝。未接受的报价不过期——没承诺就没期限。放弃订单零成本。

14 交单结算

订单满足且在窗口内时交单结算：付花瓣奖励，剪掉满足条件的盛开花（花束交付、按消耗剪花），并把指定区块地面从荒地翻成花坛。翻区块是每区块幂等的，重复交单不重复翻。交单后该区块解锁，成为可种花的花坛。

完成方法把四道否决（无此单、不在板、已过期、未满足）放在最前，任一命中零改动，通过才付奖励、剪花、翻块：

```
OrderResult CompleteOrder(string defId)
{
    OrderDef def = OrderTable.Get(defId);
    if (def == null || !IsOnBoard(defId)
        || IsExpired(defId) || !Judge(def.Target))
```

```

return new OrderResult(false, 0, Garden.State.Petals, 0);

int credited = Math.Min(WalletCap - Garden.State.Petals, def.RewardPetals);
if (credited < 0) credited = 0;
Garden.State.Petals += credited;

int flipped = 0;
int[] blockSlots = OrderTable.GetBlockSlots(def.FlipBlock);
for (int i = 0; i < blockSlots.Length; i++)
{
    //
    //
}
return new OrderResult(true, credited, Garden.State.Petals, flipped);
}

```

风险窗按 $\text{MinSlots} \times \text{目标品种生长秒数} \times 0.3 + 8\text{秒缓冲}$ 算出，双色目标取两色系里生长更慢的那个，保证留得出种花和等待的时间。判定是状态真相、不依赖是否接单，但过期的座位不会因后来满足而复活。

15 阵码分享 ShareCodec

花园可编成阵码，格式：

```
BH1|<16    >|<4    >|<    >
```

槽字符：`0` 空，`1`-`4` 对应SP01-SP04生长中，`A`-`D` 对应盛开。区块字符：`0` 荒地，`1` 已改造。花瓣数0到999。整串上限96字符。

编码逻辑：

```

//      A-D      1-4      0
if (slot.Stage == SlotStage.Bloom)
    slots[i] = (char)('A' + speciesIndex);
else if (slot.Stage == SlotStage.Growing)
    slots[i] = (char)('1' + speciesIndex);
else
    slots[i] = '0';

```

区块字符按区块首槽是否改造判定，全区块统一。分享走剪贴板和平台分享API。

16 访客解码

朋友输入阵码，DecodeShare 解码出可参观的花园。生长中的花刻意不编码种植时间，解码时 PlantedAt 归零，在访客自己的时钟上继续开放（活花园入口）。当前选中品种是本地UI状态、不编码。任何格式错误返回空、不抛异常。访客可点赞、送花束，送花是纯赠予，不做随机付费。

解码方法逐段校验，任何一段不符就返回空：

```
GardenState DecodeShare(string code)
{
    if (string.IsNullOrEmpty(code)) return null;
    string trimmed = code.Trim();
    if (trimmed.Length == 0 || trimmed.Length > MaxShareChars) return null;
    string[] parts = trimmed.Split('|');
    if (parts.Length != Sections) return null;
    if (!Equals(parts[0], Prefix, Ordinal)) return null;
    string slotSection = parts[1];
    string blockSection = parts[2];
    string petalSection = parts[3];
    if (slotSection.Length != SlotCount) return null;
    if (blockSection.Length != BlockCount) return null;

    var state = new GardenState();
    for (int i = 0; i < slotSection.Length; i++)
    {
        char c = slotSection[i];
        if (c == '0') continue;
        int speciesIndex; SlotStage stage;
        if (c >= '1' && c <= '4')
        { speciesIndex = c - '1'; stage = Growing; }
        else if (c >= 'A' && c <= 'D')
        { speciesIndex = c - 'A'; stage = Bloom; }
        else return null;
        SpeciesDef def = SpeciesTable.Get(speciesIndex);
        if (def == null) return null;
        state.Slots[i].SpeciesId = def.Id;
        state.Slots[i].Stage = stage;
        state.Slots[i].PlantedAt = 0;
    }
    for (int b = 0; b < blockSection.Length; b++)
    {
        char c = blockSection[b];
        if (c != '0' && c != '1') return null;
        if (c != '1') continue;
        int[] blockSlots = OrderTable.GetBlockSlots(b);
        if (blockSlots == null) return null;
        for (int s = 0; s < blockSlots.Length; s++)
            state.Slots[blockSlots[s]].Renovated = true;
    }
    if (petalSection.Length < 1 || petalSection.Length > 3) return null;
    int petals;
    if (!int.TryParse(petalSection, out petals)) return null;
}
```

```
if (petals < 0 || petals > WalletCap) return null;
state.Petals = petals;
return state;
}
```

每个分支的返回空都是一道防线，保证坏阵码不会拼出半个花园。

阵码状态空间信息量：16槽×9状态 + 4区块位 + 花瓣0-999，取log2，用作分享事件的信息度量。

16.1 分享页界面

ShareView 是分享覆盖层，从主界面分享按钮进入，排序层在花园和订单板之上、引导之下。界面元素按固定契约挂载：

元素	作用
阵码面	实时显示当前花园阵码
复制按钮	把阵码复制到系统剪贴板
解码输入框	输入朋友阵码
参观按钮	解码并进入访客花园
状态面	提示复制成功或阵码未识别
关闭按钮	返回花园

阵码复制走系统剪贴板，参观把解码出的花园加载成可游览状态。格式错误不抛异常，状态面提示未识别、游玩继续。分享页底部带一行 AI 辅助美术和音频的透明声明，阵码列背后衬分享卡框美术。

16.2 花园页交互

花园页 GardenView 把16槽花园程序化渲染出来，不做场景手工搭建。页面拥有GardenSystem，用单调实时时钟驱动，同时订阅花开事件。

玩家点一块空地，若当前选了品种就种下；点一朵盛开的花就收获，收获数字以“+N”飘字上浮，钱包HUD同步刷新。每个槽的花开三通道节拍相互独立、互不干扰。地块和花面通过空安全接缝加载，美术没到位时保留程序化色块面、不影响玩法。

16.3 订单板页交互

订单板页 OrderBoardView 同样程序化生成，订单判定跑在花园页拥有的同一个 GardenSystem 上，是实时判定——玩家在花园页种出的花一到位，订单板上对应订单的完成按钮立刻亮起，不用手动刷新。

每张订单卡有三个动作：接单即承诺，放弃是零惩罚中止，完成则付花瓣、把该订单的2x2区块地面在花园页翻成花坛并播放翻新金色节拍。完成反馈先剪花束、约0.25秒后再落翻地，分两拍演清楚。页面打开以及每次放弃或完成之后，订单板自动补回满板。

17 存档

SaveState 是整份存档，由五个面组成：

面	字段	内容
时钟	LastSeenAt	单调时钟锚点，离线累计基准
花园	Garden	16槽状态加花瓣钱包
订单	Orders	订单板和在途单状态
图鉴	Codex	图鉴条目列表
广告	AdCaps	广告胶囊记录

CodexEntry 字段：品种id、解锁时间、开花次数。AdCapsule 字段：类型、提供时间，广告每日12次硬上限由广告系统强制。

JSON本地存储带版本校验，SaveStore 对接平台存储API，自动保存加备份，支持导出码。恢复时给在开的槽重新盖戳，避免离线误枯萎。

存档的五个面各自独立序列化，加载时逐面校验，某一面解析失败只回退那一面、不拖垮整份进度。写盘先落临时文件再原子替换，防止游戏在保存瞬间被系统杀掉留下损坏文件。

恢复逻辑用存档里的时钟锚点重算离线，生长中和盛开的槽按规则补进度或判枯萎，结算结果透明、给玩家看到离线期间发生了什么。广告胶囊记录保证每日次数跨重登不重置，避免靠退游戏刷新广告上限。

17.1 离线结算

离线结算结果 OfflineSettlement 字段：离开秒数、原始累计、实际计入、是否封顶、结算后钱包。离线按离开秒数折算花瓣，约60秒折6个，累计封顶10个：

```
// ->
OfflineSettlement(awaySeconds, raw, accrued, capped, walletAfter)
```

原始值是未封顶的应得，计入值是实际入账，超过封顶只记封顶、Capped标记为真。回来弹离线收获摘要，显示实际计入花瓣。

18 广告

纯IAA，广告位：加速生长、收获翻倍、订单板刷新，全部可选，核心进度零广告。预加载，有频次控制，不中断操作，广告失败给跳过。

19 多语言

LocalizeService
按键取文案，全工程不硬编码中文字符串，字库预留拉丁字符位，支持中英文切换实时生效。

20 音频

AudioHost 管理, AudioStems 按场景分层混合, BGM轻柔循环, 每品种有独立花开音效, 环境层含鸟鸣风声。BGM音效分通道调音量。

21 新手引导

NewbieGuide 按逐秒表编排, 只在全新花园（没有花瓣、没有任何操作）跑一次, 回归玩家不再看到, 不做唠叨。所有引导元素射线透明, 绝不抢走玩家的操作点击。

逐秒时间窗:

时间	内容
T+0-3秒	预置的嫩芽在S01首次开花、走首个节拍
T+3-10秒	首次操作: 点被高亮的预设槽（索引1, 种子槽0旁边）种下
≤10秒	三个钩子信号: 花开节拍+首单提示+区块预览翻转动画

区块预览翻转只做视觉、展示改造后的样子（家园区块0），是改造愿景的提前演示。引导阶段：等开花、等种植、三信号、完成，每次跳转盖时间戳，供红线检查的游玩门读取。欢迎语停留1.6秒、提示停留2.5秒，文案走多语言键。

22 图鉴 Codex

CodexSystem 是花语图鉴状态机。一个品种首次开花时解锁它的图鉴条目（幂等），之后每开一次花累加该品种的开花计数，供图鉴卡片展示。

图鉴订阅花园的开花事件，离线生长走的是和在线相同的单调 Tick
路径，所以离线开的花同样解锁条目。解锁判定：

```
//
bool NoteBloom(string speciesId, double nowSeconds)
{
    SpeciesDef def = SpeciesTable.Get(speciesId);
    if (def == null) return false;
    CodexEntry entry = Find(speciesId);
    if (entry == null)
    {
        entry = new CodexEntry { SpeciesId = def.Id,
            UnlockedAt = nowSeconds, BloomCount = 0 };
        _save.Codex.Add(entry);
    }
    entry.BloomCount++;
    return true;
}
```

全部冻结品种都解锁即收集完成，未解锁品种的开花计数对外显示0，是诚实面。CodexView渲染图鉴，花语、故事和收集进度可视化。

23 容错与性能

- 阵码格式错误返回空不崩溃
- 资源缺失用程序化占位（PageArt）
- 全局异常捕获，广告失败不卡流程
- 对象池、按需加载、静态合批
- 订单和枯萎走显式扫描，不拿过期状态判定
- 包体20MB内

花园模拟吃外部时钟，种植、生长、盛开、枯萎都按时间规则走，同一串输入在任何帧率下结果一致，帧率只影响画面、不影响花开节奏。模拟不持有UI，界面订阅花开和收获事件自己渲染，结算与表现解耦。

花瓣只有一个出处，种植收获、订单奖励、离线结算都汇到同一个钱包并受上限裁剪，任何渠道都不会多发。订单按同时盛开实时判定、过期不复活，规则透明，玩家清楚要种什么、什么时候交。

阵码把花园编成可分享文本、不依赖服务器，访客在自己时钟上继续养，分享和参观都是本地完成。图鉴、订单、花园三面状态各自独立又相互对应，后期加花种和订单只在冻结表和对层扩展。

24 版本信息

项	内容
软件全称	我的小花园游戏软件
软件简称	我的小花园
版本号	V1.0
开发完成日期	2026年09月30日
发表状态	未发表
开发方式	独立开发
权利取得方式	原始取得
权利范围	全部权利
编程语言	C#
源程序量	约4400行（运行时代码）